Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Кафедра програмування

Лабораторна робота №1

РОЗПАРАЛЕЛЕННЯ ДОДАВАННЯ ТА ВІДНІМАННЯ МАТРИЦЬ

з курсу “Паралельні та розподілені обчислення”

Виконав:

студент групи ПМІ-33

Яценко Владислав Тарасович

Львів – 2025

1. Мета роботи

Мета цієї роботи – реалізувати обчислення суми та різниці двох матриць двома способами:

- послідовно, тобто один за одним елемент за елементом;
- паралельно, коли матриця розбивається на кілька частин, і кожна частина обчислюється окремим потоком.

---

2. Теоретична частина

2.1 Послідовне додавання та віднімання

Для кожного елемента ми робимо одне додавання або віднімання.
Кількість операцій: **n * m** для додавання і стільки ж для віднімання.

2.2 Паралельне додавання та віднімання

Ідея проста: розбиваємо рядки матриці на блоки, і кожен блок обчислює свій потік.

Якщо потоків k, то для кожного потоку:

```
rowsPerTask = n / k; remainingRows = n % k;
```

- Кожен потік обробляє rowsPerTask рядків.
- Якщо залишаються рядки (тобто n не кратне k), перші кілька потоків обробляють на 1 рядок більше.
- Після того, як усі потоки закінчують свою частину, програма збирає результати.

Цей момент, коли головний потік чекає завершення всіх інших потоків, називається **синхронізація**. Вона потрібна, щоб результати були правильними і щоб не виникло помилок, коли потоки ще працюють.

- **Прискорення паралельного алгоритму:**

$$S = \frac{T_{\text{посл}}}{T_{\text{парал}}}$$

- Ефективність:

$$E = \frac{S}{k} \cdot 100\%$$

Чим більше потоків, тим швидше зазвичай програма працює, але ефективність падає через накладні витрати на створення та керування потоками.

---

3. Алгоритм виконання

1. Створюємо дві матриці і заповнюємо їх випадковими числами.
2. Робимо послідовне додавання та віднімання, фіксуємо час.
3. Робимо паралельне додавання та віднімання:
    - ділимо матрицю на блоки рядків;
    - запускаємо для кожного блоку окремий потік;
    - чекаємо завершення всіх потоків (синхронізація).
4. Обчислюємо прискорення та ефективність.

---

4. Результати

1. Для матриць розмірності 15000x15000 та кількістю потоків k = 2 результати вийшли такими:

```
Час затрачений на ПОСЛІДОВНЕ додавання: 1161ms
Час затрачений на ПАРАЛЕЛЬНЕ додавання: 623ms
Прискорення (додавання): 1,86x
Ефективність (додавання): 93,18 %

Час затрачений на ПОСЛІДОВНЕ віднімання: 1063ms
Час затрачений на ПАРАЛЕЛЬНЕ віднімання: 623ms
Прискорення (віднімання): 1,55x
Ефективність (віднімання): 77,48 %
```

2. Для матриць розмірності 15000x15000 та кількістю потоків k = 8 результати вийшли такими:

```
Час затрачений на ПОСЛІДОВНЕ додавання: 1115ms
Час затрачений на ПАРАЛЕЛЬНЕ додавання: 274ms
Прискорення (додавання): 4,07x
Ефективність (додавання): 50,87 %

Час затрачений на ПОСЛІДОВНЕ віднімання: 1103ms
Час затрачений на ПАРАЛЕЛЬНЕ віднімання: 274ms
Прискорення (віднімання): 3,60x
Ефективність (віднімання): 45,06 %
```

5. Аналіз результатів

- Паралельне обчислення швидше за послідовне.
- Час зменшується зі збільшенням потоків, але ефективність падає через накладні витрати на управління потоками.
- Якщо рядків не кратно потокам, програма все одно обробляє всі рядки, завдяки додатковим рядкам, які діляться між першими потоками.
- Оптимальна кількість потоків приблизно дорівнює числу логічних ядер процесора — тоді прискорення максимально ефективне.

Висновок

Виконуючи завдання, я навчився прискорювати обчислення матриць за допомогою паралельного програмування та правильно розподіляти роботу між потоками.